Practice III — Product Catalog & Email App

Logan Stum

Source code: https://github.com/logan-stum/practice3

Key Design Decisions

One of the main design choices in this project was using a single ProductAdapter for both activities instead of maintaining separate adapters. The adapter accepts a showCheckbox flag that determines whether selection checkboxes should appear. This allowed both screens to reuse the same binding logic and row layout (item_product.xml) while keeping the implementation simpler and easier to maintain.

For passing selected products between activities, the application sends complete Product objects through the Intent rather than only passing database IDs. Since the selected products are already available in memory, SecondActivity can directly display them without performing another database query or creating another dependency on DBHelper. This keeps the second screen focused strictly on displaying and emailing the selected items.

The application uses ACTION_SENDTO with a mailto: URI when launching the email client. I chose this approach because it limits the chooser dialog to email applications only. Using a more general intent such as ACTION_SEND would include unrelated apps like messaging or social media platforms, which did not align with the assignment requirements.

Another design consideration was separating the RecyclerView state from the database itself. When the user completes the email flow, the clearAll() method only removes products from the adapter’s in-memory list. The SQLite database is left

unchanged so the original product catalog remains intact, which matches the assignment requirement that products disappear from the current view only.

To make testing easier, the database is pre-seeded with eight sample products during the first launch of the application. This allows the grader to immediately test product selection and email functionality without needing an additional product-entry screen.

Challenges

One issue I encountered was handling email completion reliably. Different email clients behave differently when launched through an implicit intent, and many do not consistently return RESULT_OK after a message is sent. Because of this, relying on the result code was not dependable. Instead, the application uses registerForActivityResult and treats the user returning to SecondActivity as the completion point for displaying the confirmation Toast and clearing the displayed list.

Another challenge involved keeping checkbox selections synchronized with the RecyclerView. Since RecyclerView reuses item views while scrolling, checkbox listeners could sometimes trigger for the wrong product if they were not reset correctly during binding. To address this, listeners are temporarily removed before updating checkbox state and then reassigned afterward, preventing recycled views from carrying over stale callbacks.

Passing product objects between activities also required some adjustment. The Product class needed to implement Serializable so the selected product list could be attached directly to the Intent. I also had to ensure the class did not contain non-

serializable fields such as Bitmap objects, otherwise the transfer between activities would fail.